Регламент практик по ООП 2026

Цели студентов на практике по ООП:

1. Научиться детальному проектированию и программированию в объектно-ориентированном стиле. Используя обратную связь от преподавателя, улучшить своё понимание ООП.
2. Попрактиковаться в командной работе над проектом. В частности, научиться общаться про задачи и планировать совместную работу. А также научиться простым сценариям работы с git.
3. Освоить язык Java и базовые технологии, доступные на этой платформе.
4. Создать проект для своего портфолио.

Задачи

В течение семестра предстоит создать и развивать один проект — чат-бота на произвольную тему. Например, задающего пользователю задачки / загадки / вопросы и оценивающего ответы. Или бота, предоставляющего доступ к какому-либо API.

Стартовая задача

При старте диалога, чат-бот рассказывает, что он умеет и как с ним взаимодействовать. Затем в бесконечном цикле задает вопрос, получает от пользователя ответ и оценивает его правильность, задаёт следующий вопрос и так далее. В любой момент можно командой \help попросить бота ещё раз рассказать о себе, и пояснить как с ним взаимодействовать.
Общаться с ботом можно через консоль. К организации кода предъявляются следующие требования:

1. Код, реализующий хранилище вопросов или их генератор, должен находиться в отдельном классе от кода, реализующего логику диалога. Причем классы, реализующие хранилище / генератор вопросов не должны зависеть от классов, реализующих логику диалога.
2. Аналогично по разным классам должен быть разделен код, реализующий логику диалога и код, работающий с консолью. Причем классы логики не должны зависеть от классов с консолью.

Тематику вопросов, вид ответов и способ оценивания ответов студенты придумывают сами. Начать стоит с чего-то максимально простого.

Эта и все следующие задачи должны быть покрыты модульными тестами.

Последующие задачи

Начиная со второй задачи, к архитектуре бота предъявляются два дополнительных требования:

1. Она должна обязательно поддерживать ведение множества диалогов одновременно с разными пользователями. Рекомендуется заложить подходящую для этого архитектуру ещё при выполнении первой задачи.
2. Она должна подходить для реализации ботов для типичных чат-платформ: Телеграма, Алисы, Вконтакте. Обычно чат-платформы присылают вашему коду очередную реплику пользователя и ждут в ответ одну реплику бота. То есть у чат-платформы нет метода, аналогичному new Scanner(System.in).nextLine(), позволяющему “вытянуть” очередную реплику из пользователя. Можно только “среагировать” на пришедшую реплику.

Начиная со второй задачи в течение семестра студенты сами выбирают новые возможности, которые собираются добавить в чат-бота. Всю работу над проектом студенты делят на *задачи*. Преподаватели оценивают каждую задачу независимо.

Задача — это объем работ с четко сформулированной одной целостной целью, которую можно объяснить за 2 минуты. Формулировка задачи должна быть зафиксирована письменно, не требовать дополнительных пояснений и по объему текста не превышать размер этого абзаца. Задача должна быть интересна с точки зрения проектирования — предполагать существенную доработку или переработку текущей архитектуры. Рекомендованный ориентировочный объем работы на одну задачу — 200 строк рабочего кода, 2–5 классов, модульные тесты, ~ 4 часа работы.

Студенты предлагают свои формулировки задачи. Если задача добавляет возможностей боту, то формулировки должны снабжаться 1-3 примерами диалогов, которые станут возможны после этой задачи. Преподаватель корректирует формулировку, чтобы сделать задачу более полезной с точки зрения обучения: предлагает дополнительные требования, чтобы пришлось применить приёмы проектирования, или советует, как сократить объем работы для студентов.

Сдать задачу можно только после того, как преподаватель явно согласился с её формулировкой. Валидировать формулировки задач лучше через чат группы в телеграмме.

Примеры формулировки задачи

Пример 1. Инфраструктурная задача

Сделать так, чтобы бот одновременно является и ботом для Телеграма и навыком для Алисы. Логику чат-бота отделить от специфики Телеграма и Алисы
В чем сложность и интерес:

1. Сделать удобный инфраструктурный код для создания чат-ботов, одновременно подходящих для нескольких чат-бот-платформ.
2. Поддержать уникальные возможности каждой из платформ: разметка ударений для Алисы и разметка html для Телеграма.

Пример 2. Задача про диалог

Реализовать игру в города с подсказками. Пример диалога:
П: Начинаем!
Б: Казань
П: Киров
Б: Эй! Нужно называть город на последнюю букву предыдущего, на которую могут начинаться города. Конкретно сейчас нужен город на букву Н.
П: Нск
Б: Нет такого города!
П: Новосибирск
Б: Курск
П: Подскажи!
Б: Есть один город. На Ки начинается, на в заканчивается…
П: Сдаюсь!
Б: Ура! Я победил! Ещё разик?

Теория

Если для выполнения задачи студентам требуются знания, ещё не пройденные на лекциях, студенты самостоятельно осваивают эту тему по лекционным слайдам или по аудиоверсии курса. Они доступны по ссылке:
https://courses.insma.urfu.ru/oop/java_oop_2_java_cpp/

Работа в парах

Каждая задача делается в паре с другим студентом.

Результат работы одного студента в паре должен проходить ревью другим студентом пары. Каждый в паре должен нести ответственность за весь код задачи, уметь объяснить и защитить каждое принятое решение.

Есть несколько шаблонов работы в паре. Выбирайте!

1. Вы весь код пишите в паре, собравшись в одном месте физически. Пишите одновременно код и модульные тесты на него. Так оба будут знать всё про задачу.
2. Вы декомпозируете задачу на подзадачи и каждый занимается своей частью. Каждый пишет и код и тесты для своего кода. В конце интегрируете результаты во что-то рабочее.
3. Не рекомендуется делиться по принципу: один пишет код, а второй тесты. В реальной работе разработчик пишет модульные тесты на свой код самостоятельно, в идеале параллельно с написанием кода (TDD).

В любой момент студенты могут поменять пару. В любой момент преподаватель может переформировать пары другим способом. Новая пара студентов выбирает одну из двух кодовых баз, создают её копию (fork на языке github) и продолжают работать над ней. Может быть так, что две пары после обмена участниками продолжат работу над одной и той же кодовой базой. Это нормально.

Для распределения по парам, сообщите о своем выборе преподавателю практики. Если не можете сами найти пару, попросите преподавателя помочь. Также узнайте у преподавателя, где можно смотреть ваши текущие результаты, как получать обратную связь.

Git

Весь исходный код должен быть в системе контроля версий git. Рекомендованная система — https://github.com, но разрешено использовать и аналоги, например https://gitlab.com/ или https://bitbucket.org/.

Как освоиться с гитом? От простых способов к сложным:

1. Пройти краткий гайд
2. Пройти интерактивные учебные курсы от github и schoolacademy.
3. Прочитать официальную книгу по git: http://git-scm.com/book/ru/v2 Первые три главы обязательны для уверенного использования git.

Снизить количество ошибок при использовании гита можно, если для операций коммита и просмотра изменений использовать один из графических клиентов. Подойдут, Git Extensions, TortoiseGit (только для Windows), GitHub Desktop или SourceTree.

Важно соблюдать гигиенические требования при пользовании системы контроля версий:

1. Git-репозиторий должен быть самодостаточным. Скачав исходники в новую директорию на чистый компьютер должно быть возможно собрать и запустить весь ваш проект. Не должно быть файлов, присутствующих только на вашей рабочей машине. Иначе вашему напарнику будет тяжело, а виноваты в этом будете вы.
2. Git-репозиторий не должен содержать ничего лишнего. Файлы, которые появляются в процессе компиляции, файлы с пользовательскими настройками вашей IDE, бинарные зависимости (если они получаются менеджером пакетов, например, maven) — все это должно быть включено в .gitignore-файл, и не должно попадать в репозиторий.
3. Текстовые сообщения к коммитам должны быть содержательными (то есть по их тексту должно быть ясно, что именно сделано в данном коммите).
4. Перед коммитом просматривайте все те изменения, которые вы совершили и отменяйте лишние изменения (обычно это легко сделать с помощью графического клиента). Например, если вы сначала решили изменить класс X, а потом передумали и всё отменили, то лучше откатить вообще все изменения в этом

файле. Это даст вашему напарнику лучшее представление о том, что вы меняли, а что не собирались.
5. Перед коммитом проверяйте работоспособность всего проекта.
6. В git-репозитории не должно быть никаких секретных ключей или паролей.
7. В git должна быть видна работа над задачами *обоих* участников пары.

Формулировки задач также стоит сохранять в git. Это удобно делать в файле Readme.md в корне репозитория, потому что содержимое этого файла показывается на главной странице вашего репозитория, а редактировать его можно прямо через браузер.Также во всех названных системах для работы с git есть возможность явно заводить задачки (они обычно называются issue), там можно и подробно описать задачу, и затем отслеживать ее решение. Удобно в коммите делать ссылку на номер задачи.

Замечания преподавателя также нужно фиксировать там же, рядом с формулировкой задания.

Код, демонстрируемый преподавателю должен быть закоммичен. Нельзя сдавать код, который ещё не в репозитории.

Баллы

За семестр можно сдать 6 задач.

Каждая задача в случае успешной сдачи оценивается в 2 балла, если она выполнена в целом хорошо; в 1 балл, если остались серьезные проблемы, либо если один из студентов пары не понимает существенной части решения задачи; либо не оценивается, если по мнению преподавателя, она не готова — тогда её можно будет сдать на следующей паре.

Одинаковые баллы получают оба напарника.

На каждом занятии можно попытаться сдать только одну задачу. Сдавать задачу можно только после того, как преподаватель принял вашу формулировку задачи или скорректировал её. Ожидайте, что с первого раза сдать задачу не получится — будут претензии к качеству и доработки, которые вы сможете сдать только на второй раз.

Одну задачу нельзя сдавать более трех раз. После третьей попытки её сдать преподаватель ставит либо 0, либо 1 балл по своему усмотрению.

Рекомендуется готовить формулировку следующей задачи заранее и согласовывать её с преподавателем либо через чат группы до пары, либо сразу после сдачи предыдущей задачи.

Правильно придумывая и проектируя задачи можно набрать ещё несколько дополнительных баллов следующими способами:

1. (+2) В любой момент после успешной сдачи третьей задачи можно продемонстрировать репозиторий в git и историю коммитов. Если качество работы с git удовлетворительное (см гигиенический минимум по git-у), то можно заработать 1 или 2 дополнительных баллов.
2. (+2) Начиная со второй задачи за идеальное качество unit-тестов по задаче, но только один раз. Возможность получения этого допа нужно заранее уточнить у преподавателя. Одновременно с этим обсудить, какие тесты он бы ожидал по данной задаче.
3. (+2) За оформление чат-бота в виде бота для Телеграм или навыка для Яндекс Алисы или для любой другой популярной платформы для создания чат-ботов. После сдачи очередной задачи, нужно продемонстрировать преподавателю работоспособность бота через интерфейс телеграмма или Алисы.
    a. (+2) За использование особых возможностей чат-платформы: кнопки, разметка, изображения в телеграмме, кнопки и ударения в словах в навыке Алисы.
4. (+2) За техническую сложность задачи. Для этого студенты должны заранее согласовать с преподавателем объем работ для этого допа. Рекомендуется не пытаться брать этот доп до третьей задачи.
5. (+1) За деплой вашего телеграмм-бота или навыка для Алисы в интернет. Azure, Heroku, Amazon, … Для получения этого балла, вам нужно продемонстрировать в Readme.md файле репозитория инструкцию по развертыванию приложения на одной из облачных платформ (на ваш выбор) и продемонстрировать работоспособность развернутого в облаке бота.
6. (+1) За использование внутри чат-бота любого внешнего источника данных или API (кроме Telegram и Алисы). API github, почты России, VK, ...

Чтобы получить дополнительные баллы за задачу, нужно явно сформулировать это желание преподавателю, например так: «Мы считаем, что в этот раз написали идеальные unit-тесты и хотим получить дополнительные баллы»

Доп-баллы нельзя получить на двух последних парах в семестре.

Границы баллов:

1. 20 баллов (из 24 возможных) — это эквивалент 100 баллов в системе БРС
2. Ориентировочный уровень «отлично» — 16 баллов
3. Для допуска к зачёту достаточно 10 баллов
4. Предварительная граница для упрощённой сдачи зачёта: 16 баллов

Примеры идей для задач

Студенты *не* ограничены только этими идеями, приветствуются собственные, оригинальные идеи.

Идеи специально даны общо и не являются формулировками задач. Если за основу взята одна из идей, то формулировку студенты должны составить самостоятельно, уточнив состоит сложность и интересность с точки зрения проектирования, или чему предполагают научиться на этой задаче.

1. Телеграм-бот / Навык для Алисы
2. Конечный автомат для реализации диалогов.
3. Возможность проиграть.
4. Разные виды вопросов. На точность, на скорость, открытые / закрытые, с самоконтролем пользователя.
5. Перевод игрока на повышенный уровень сложности, если он хорошо играет.
6. Возможность у пользователя узнать статистику по своим ответам.
7. Сохранение всех событий в облачной базе данных (mongo / firebase)
8. Админка для редактирования набора вопросов.
9. Усложнение структуры диалога: возможность выбрать тематику вопросов / уровень сложности, попросить подсказку или объяснения.
10. Добавить возможность работы в чатах (для Телеграм-ботов)
11. Админка для просмотра статистики использования пользователей.
12. Тонкий клиент. Веб-интерфейс для работы с ботом через браузер.
13. Толстый клиент. Графическое приложение, через которое можно общаться с ботом.
14. Инициация диалога самим ботом. «Давно не играли. Сыграем?»
15. Алгоритм повторения вопросов, на которые пользователь не смог ответить.
16. Архитектура для декларативного описания диалогов.
17. Возможность пользователя пожаловаться на плохой вопрос / неправильный ответ.
18. Возможность дуэлей с другими пользователями: игры на одинаковых вопросах.

Для поиска идей, можете изучать каталоги и рейтинги существующих чат-ботов в Телеграме и Алисе:
http://telegram.org.ru/telegram-bots/
https://dialogs.yandex.ru/store/